

— SPECS ANTES QUE PROMPTS

Spec-Driven Development para Agentic Coding.

Especificar primero, programar después.

Angelo Leva | Senior Software Engineer • Tranzact / Codinglatam



De la conversación caótica al artefacto de ingeniería.

- **Vibe coding** — qué es y por qué no escala.
- **Spec-Driven Development** — el cambio de paradigma.
- **El flujo SDD** — Constitution, feature loop y herramientas.
- **Tu rol cambia** — de escribir prompts a orquestar.

- CAPÍTULO 01 / 04

Vibe coding no escala.

Escribes un prompt, esperas lo mejor e iteras **a ciegas**.

agent - chat

prompt 1

> "hazme un botón"

prompt 2

> "más pequeño, redondeado"

prompt 3

> "verde, no ese verde..."



RESULTADO

Conversación larga,
historial perdido,
código desechable.

deuda técnica ↑



De esa conversación con el agente queda guardado como artefacto del proyecto.

Funciona para un botón.

No funciona para un proyecto de varios meses con varias personas.

prompt → resultado → 

- CAPÍTULO 02 / 04

Spec-Driven Development.

El qué y el por qué se separan del cómo.

SPECIFICATION

● ● ● spec.md

```
> Create button:  
- color: brand green  
- corners: rounded  
- text: "Button"  
- font color: white  
- shade: same as button...
```



IMPLEMENTACIÓN

Button

```
// generado por el agente  
// a partir de la spec
```

“

La spec es un contrato –
entre humanos, y entre el humano y el
agente.

– Spec-Driven Development, definición de trabajo

“

La habilidad más importante
es escribir specs que capturen
tu intención y tus valores.

— Sean Grove (OpenAI) • AI Engineer Conference

Las **especificaciones** — no los prompts ni el código — se están volviendo la unidad fundamental de la programación.

Lo que cambia cuando pones la **spec primero**.

	VIBE CODING	SPEC-DRIVEN DEVELOPMENT
Punto de partida	Prompt suelto — "hazme esto"	Spec acordada — qué y por qué
Artefacto que persiste	El código (a veces) y nada más	Spec viva + código + tests
Cuando algo cambia	Otro prompt, otro intento	Editas la spec y la vuelves a generar
Escala con el equipo	No — el contexto vive en tu cabeza	Sí — la spec es el contrato compartido

Convertir intenciones en specs.

INTENCIONES (RUIDO)

"quiero una web app"

¿TypeScript?

¿colores / diseño?

usuarios + citas

"un botón..."

¿quién la usa?

casos borde...



SPECIFICATION (SEÑAL)

constitution.md

```
> Define project requirements
target audience: ...
stack: TypeScript
color scheme: ...
```

Phase 1: database schema

- Table 1: users
- Table 2: appointments

Por qué la spec gana cuando trabajas con **agentes**.

01 - CONTROL

Cambios pequeños en la spec, cambios grandes en el código.

Dos frases sobre la apariencia y el estilo se traducen en cientos de líneas de CSS. Menos carga cognitiva.

02 - PERSISTENCIA

Eliminas el context decay entre sesiones.

La ventana de contexto del agente se llena y empieza a fallar. La spec sobrevive a la sesión, al agente y al equipo.

03 - FIDELIDAD

El código se parece más a lo que querías construir.

La spec te obliga a definir el problema, los criterios de éxito, las restricciones y los flujos antes de que el agente empiece.

En código existente, un prompt suelto produce código que funciona pero no encaja.

SIN SPEC

El agente improvisa.

- Elige un state management distinto al tuyo.
- Recrea funcionalidad que ya existía.
- Ignora restricciones que nadie escribió.

CON SPEC

La spec carga el contexto al frente.

- **Qué** construir y cómo integra con lo que existe.
- Patrones a seguir, restricciones a respetar.
- El conocimiento implícito se vuelve explícito.

La Constitution fija las decisiones del proyecto.

CONSTITUTION

01 Mission

02 Tech Stack

03 Roadmap

Mission

¿Cuál es el por qué? Visión, audiencia y alcance del proyecto.

Tech Stack

¿Cómo encaja con la tecnología? Restricciones del equipo.

Roadmap

¿Qué funciones están planeadas? Documento vivo, por fases.

La Feature Spec traduce una idea en algo construible.

01 · SCOPE

Qué se construye.

El alcance de la función. Qué entra y qué no entra. Sin ambigüedad.

02 · REQUIREMENTS

Restricciones que importan.

Versiones fijas y decisiones técnicas no negociables. Sin nombres de variables.

03 · VALIDATION

Cómo se sabe que está listo.

Criterios de éxito que el agente puede verificar por sí mismo.

El agente entrevista, propone y hace preguntas. **Tú decides** — no aceptes lo que no te convence; aclara lo que te incomoda.

```
// los cambios aquí
// se expanden en cientos
// de líneas de código
tiempo bien invertido.
```

El agente ejecuta. Tú eres el **arquitecto** que supervisa.

Contexto limpio antes de empezar.

Usa `/clear` para que el agente empiece desde la spec, no desde residuos de conversaciones anteriores.

Grupos de tareas, no todo de golpe.

Para áreas críticas — seguridad, base de datos — implementa grupo por grupo. Los errores pequeños no se acumulan.

Observa en tiempo real.

Sigue los cambios en consola mientras el agente trabaja. Revisa el diff antes de que termine.

EL ROL DEL DESARROLLADOR

Proveer un contrato claro. No microgestionar el código.

El agente valida su propio trabajo al terminar. La revisión humana viene después — esa es la siguiente etapa.

```
// flujo de implementación
/clear ← contexto limpio
implement tasks ← ejecutar
review diff ← supervisar
commit ← cerrar
```

Revisa la **intención**, no el detalle.

Revisa el commit view.

¿La función refleja la spec? ¿Cumple los criterios de éxito? No te detengas en qué clases CSS se usaron.

Si algo está mal en el código, busca el error en la spec.

Un error en la implementación muchas veces viene de un error en el plan. Corrígelo en los dos lugares a la vez.

Cuidado con el drift.

Si mueves un archivo manualmente, deja que el agente actualice specs, READMEs y referencias.

DEUDA COGNITIVA

**Los agentes son rápidos.
Los cambios se acumulan.**

Para reducir la carga mental: cambios manejables, commits frecuentes y spec siempre sincronizada con el código.

AL TERMINAR

Merge + actualiza la Constitution.

El roadmap registra la función como completada. La spec y el código avanzan juntos en la misma rama.

Después de cada función, replantea.

01 • ACTUALIZA

La Constitution es un documento vivo.

Nuevas decisiones técnicas, preferencias de tests, diseño responsivo — actualiza la spec antes de seguir programando.

02 • REVISA EL ROADMAP

¿El próximo paso sigue teniendo sentido?

Con lo aprendido, reagrupa funciones o ajusta prioridades. El roadmap también evoluciona.

03 • AUTOMATIZA

Empaqueta el proceso repetible en skills.

Validación, changelogs, lint, tests — lo que haces en cada merge puede convertirse en un skill del agente.

Replantear no es solo sobre una función — es sobre mejorar **cómo trabajas** con el agente en todos tus proyectos.

Cambio pequeño → **implementalo ahora.**

Cambio grande → agrégalo al roadmap como su propia fase.

Usa tu criterio. La spec captura decisiones, no solo código.

Cuatro comandos. Una secuencia.

Puntos de revisión humana.

`/specify`

Qué y por qué.

Historias de usuario,
requisitos y criterios de éxito.
Sin stack técnico.

`/plan`

**Cómo,
técnicamente.**

Stack, arquitectura, contratos
y archivos impactados.

`/tasks`

**Lo que hay por
hacer.**

Tareas atómicas, ordenadas
por dependencia, marcadas
en paralelo.

`/implement`

Ejecución.

El agente programa contra
spec + plan + tasks. Tú
revisas diffs acotados.

`specify` → (revisión) → `plan` → (revisión) → `tasks` → `implement`

// human-in-the-loop en cada punto de revisión

El agente no escribe código antes que los **tests**.

01 · RED

Tests primero.

El agente escribe los tests a partir de la spec. Tú los apruebas. Tienen que **fallar**.

02 · GREEN

Implementación mínima.

Recién ahora se escribe código — lo mínimo para que pasen los tests aprobados.

03 · VERIFY

Contra la spec.

Se valida que la implementación cumple la spec, no solo los tests. Detección de issues y casos borde.

Invierte el patrón clásico de generación con IA: en lugar de generar código y cruzar los dedos, el agente genera **tests que definen el comportamiento**, los aprobamos y solo entonces implementa.

Git. Ship. Done.

Un loop de **cinco fases** contra el context rot.

01 · DISCUSS

Decidir antes de planear.

Se capturan las decisiones de implementación antes de que exista un plan.

02 · PLAN

Investigar y descomponer.

Se verifica que el plan quepa en una ventana de contexto fresca.

03 · EXECUTE

Oleadas en paralelo.

Cada ejecutor arranca con su propio contexto limpio, no con el tuyo gastado.

04 · VERIFY

Recorrer lo construido.

Se diagnostica y se corrige antes de declarar la fase terminada.

05 · SHIP

PR y archivar.

Se abre el PR, se archiva la fase y el loop empieza de nuevo.

●●● gsd-core

```
# instalación, agnóstica del runtime
$ npx @opengsd/gsd-core@latest
```

```
> /gsd-new-project # proyecto nuevo
> /gsd-onboard     # repo existente
```

EL TRABAJO PESADO VA AFUERA

Investigación, planificación y ejecución corren en subagentes con contexto fresco.

EL ESTADO SOBREVIVE

STATE.md y **CONTEXT.md** son artefactos del repo que la siguiente corrida carga.

Un framework ligero para llevar la spec dentro del repositorio.

INTENCIÓN

Revisa intención, no solo código.

Cada cambio produce un **spec delta**: qué requisito cambia, no qué línea cambia.

PERSISTENCIA

Contexto que persiste.

Las specs viven en el repo junto al código.
No desaparecen cuando termina la sesión o el desarrollador se va.

VELOCIDAD

Algo que revisar en segundos.

Antes de escribir una sola línea de código tienes: propuesta, decisiones técnicas, tareas y spec deltas.

```
npm install -g @fission-ai/openspec@latest
```

Universal • Open Source • Sin API keys • Sin MCP

Un comando genera todo lo que necesitas revisar **antes de programar.**

● ● ● openspec

```
# describes el cambio que quieres  
> /openspec:proposal Add remember me  
checkbox with 30-day sessions
```

```
# lee las specs existentes y el código,  
# luego genera:
```

```
openspec/changes/add-remember-me/  
├─ proposal.md ← qué cambia  
├─ design.md   ← decisiones técnicas  
├─ tasks.md    ← implementación  
└─ specs/      ← spec deltas
```

BROWNFIELD-FIRST

Pensado para bases de código maduras, no proyectos nuevos. Las specs se crean a medida que construyes.

UNIVERSAL

Funciona con Claude Code, Cursor, Copilot, Gemini CLI y 20+ herramientas más. Las specs no dependen del agente.

NO ES PLAN MODE

Plan mode vive en una sesión. OpenSpec vive en el repo — sobrevive sesiones, agentes y equipos.

- CAPÍTULO 04 / 04

**Tu rol
cambia.**

Tú eres el **orquestador**. El agente, el que ejecuta.

• TÚ • ORQUESTADOR

Diseñas. Supervisas. Apruebas o pides cambios.

Aportas el contexto que el agente no tiene: el por qué, las restricciones y los criterios de éxito.

especificar

revisar

guiar

aprobar

• AGENTE • PROGRAMADOR

Conoce el stack técnico, tiene velocidad y baja al detalle.

Toma la spec como plano y razona la implementación. Tiene acceso al repositorio y a las herramientas.

planear

programar

refactorizar

testear

SDD y vibe coding no son **excluyentes**.

VIBE CODING SIRVE CUANDO...

- Es un prototipo desechable.
- Quieres explorar una idea rápido.
- El alcance entra en una sola sesión.
- El código no va a producción.

SDD SIRVE CUANDO...

- Es una función de producción.
- Participa más de una persona.
- El proyecto va a vivir más allá de esta semana.
- Hay restricciones que no puedes perder.

Cada herramienta para su trabajo.

— PARA LLEVARTE

Specs antes
que prompts.
Ingeniería
antes que vibes.

El agente es el músculo • la spec, el cerebro | Gracias.

